

UNIDADE 1 - O MODELO RELACIONAL

O modelo relacional foi proposto por Edgar F. Codd em 1970 e organiza os dados em relações, que na prática são as tabelas. Cada tabela é composta por linhas, chamadas de tuplas, e por colunas, chamadas de atributos. Um atributo tem um domínio, que é o conjunto de valores que ele pode assumir: um atributo de domínio inteiro não aceita texto, e um atributo de domínio data não aceita números arbitrários.

A chave primária é o atributo ou o conjunto de atributos que identifica cada tupla de forma única dentro da tabela. Ela não pode ser nula e não pode repetir. Quando nenhum atributo natural serve como identificador confiável, o projetista costuma criar uma chave substituta, geralmente um número sequencial ou um identificador universal.

A chave estrangeira é o atributo que aponta para a chave primária de outra tabela. É ela que materializa o relacionamento entre duas tabelas. A integridade referencial é a regra que garante que toda chave estrangeira aponte para uma tupla que realmente existe. Se um pedido faz referência ao cliente de número 42, o banco impede que esse cliente seja apagado enquanto o pedido existir, ou propaga a exclusão, dependendo da política configurada.

O valor nulo merece atenção especial. Nulo não é zero e não é string vazia: nulo significa ausência de informação. Por isso comparações com nulo não retornam verdadeiro nem falso, mas desconhecido, e é por isso que se usa IS NULL em vez de igual a nulo.

UNIDADE 2 - NORMALIZACAO

Normalizar e reorganizar as tabelas para eliminar redundancia e as anomalias que a redundancia provoca. As anomalias sao de tres tipos. A anomalia de insercao ocorre quando nao e possivel registrar um fato sem registrar outro que ainda nao aconteceu. A anomalia de atualizacao ocorre quando o mesmo dado esta repetido em varias linhas e a alteracao de apenas algumas delas deixa a base inconsistente. A anomalia de exclusao ocorre quando apagar uma linha faz desaparecer uma informacao que nao tinha relacao com ela.

A primeira forma normal exige que todos os atributos sejam atomicos, ou seja, que nenhuma coluna guarde lista ou grupo repetido. Uma coluna telefones contendo tres numeros separados por virgula viola a primeira forma normal.

A segunda forma normal exige que a tabela esteja na primeira forma normal e que todo atributo nao chave dependa da chave primaria inteira, e nao apenas de parte dela. Essa regra so tem efeito pratico quando a chave primaria e composta por mais de um atributo.

A terceira forma normal exige que a tabela esteja na segunda forma normal e que nenhum atributo nao chave dependa de outro atributo nao chave. Essa situacao e chamada de dependencia transitiva. Se a tabela de funcionarios guarda o codigo do departamento e tambem o nome do departamento, o nome depende do codigo, e nao da chave da tabela: e uma dependencia transitiva, e o nome do departamento deveria morar na tabela de departamentos.

Normalizar ate a terceira forma normal e o padrao da maioria dos projetos. Existe desnormalizacao deliberada em cenarios de leitura intensa, quando o custo das juncoes pesa mais do que o custo da redundancia controlada, mas essa e uma decisao consciente de desempenho, nao um descuido de modelagem.

UNIDADE 3 - LINGUAGEM SQL

A SQL se divide em sublinguagens. A DDL, linguagem de definicao de dados, cria e altera estruturas, com comandos como CREATE, ALTER e DROP. A DML, linguagem de manipulacao de dados, mexe no conteudo, com INSERT, UPDATE, DELETE e SELECT. A DCL, linguagem de controle de dados, cuida de permissoes, com GRANT e REVOKE.

A consulta SELECT segue uma ordem logica de avaliacao diferente da ordem em que e escrita. Primeiro o banco resolve o FROM e as juncoes, depois filtra as linhas com o WHERE, depois agrupa com o GROUP BY, depois filtra os grupos com o HAVING, depois projeta as colunas do SELECT e por fim ordena com o ORDER BY. Essa ordem explica um erro comum: nao se pode usar no WHERE um apelido criado no SELECT, porque no momento em que o WHERE e avaliado aquele apelido ainda nao existe.

A diferenca entre WHERE e HAVING vem dessa mesma ordem. O WHERE filtra linhas individuais antes do agrupamento; o HAVING filtra grupos ja formados e por isso pode usar funcoes de agregacao como SUM, COUNT e AVG.

As juncoes combinam tabelas. O INNER JOIN devolve apenas as linhas que casam nos dois lados. O LEFT JOIN devolve todas as linhas da tabela da esquerda e completa com nulo o que nao casa na direita. O RIGHT JOIN faz o inverso. O FULL JOIN devolve tudo dos dois lados. Um erro classico e usar LEFT JOIN e depois filtrar no WHERE uma coluna da tabela da direita: o filtro descarta as linhas com nulo e o resultado passa a ser equivalente a um INNER JOIN.

UNIDADE 4 - TRANSACOES E PROPRIEDADES ACID

Uma transacao e um conjunto de operacoes tratado como uma unidade indivisivel de trabalho. Ou tudo acontece, ou nada acontece. O exemplo classico e a transferencia bancaria: debitar de uma conta e creditar em outra precisam acontecer juntos, porque metade da operacao deixaria dinheiro desaparecido.

As quatro propriedades das transacoes formam a sigla ACID. A atomicidade garante que a transacao seja tudo ou nada, e e implementada com o COMMIT, que confirma, e o ROLLBACK, que desfaz. A consistencia garante que a transacao leve o banco de um estado valido para outro estado valido, respeitando todas as restricoes declaradas. O isolamento garante que transacoes simultaneas nao interfiram umas nas outras de forma indevida. A durabilidade garante que, depois do COMMIT, o efeito sobrevive a uma queda de energia, o que na pratica e obtido gravando primeiro em um log de transacoes.

O isolamento tem niveis, porque isolamento total custa caro em desempenho. No nivel READ UNCOMMITTED uma transacao pode ler dados ainda nao confirmados por outra, o que produz leitura suja. No nivel READ COMMITTED so se le o que ja foi confirmado, mas duas leituras iguais dentro da mesma transacao podem devolver valores diferentes, o que se chama leitura nao repetivel. No nivel REPEATABLE READ o mesmo dado lido duas vezes devolve o mesmo valor, mas novas linhas inseridas por outra transacao ainda podem aparecer, fenomeno chamado de leitura fantasma. No nivel SERIALIZABLE o resultado equivale ao de execucao sequencial, e nenhum desses fenomenos ocorre.

Deadlock acontece quando duas transacoes esperam uma pelo recurso que a outra segura. O banco detecta o ciclo e escolhe uma das transacoes como vitima, desfazendo o trabalho dela para que a outra prossiga.

UNIDADE 5 - INDICES E DESEMPENHO

Um índice é uma estrutura auxiliar que acelera a busca, ao custo de espaço em disco e de escrita mais lenta. A analogia útil é o índice remissivo de um livro: encontrar um assunto pelo índice é muito mais rápido do que folhear o livro inteiro, mas o índice ocupa páginas e precisa ser refeito a cada edição.

A estrutura mais comum é a árvore B, que mantém os dados ordenados e permite busca, inserção e remoção em tempo logarítmico. Ela serve tanto para busca por igualdade quanto para busca por faixa de valores e para ordenação. O índice hash, por outro lado, é muito rápido para busca por igualdade exata, mas não serve para faixa nem para ordenação, porque a função de espalhamento destrói a ordem original.

Nem todo índice ajuda. Um índice sobre uma coluna de baixa cardinalidade, como um campo que só aceita ativo ou inativo, raramente compensa, porque o banco vai ler quase a tabela inteira de qualquer jeito. Aplicar uma função sobre a coluna no WHERE também costuma anular o índice, porque o valor indexado é o da coluna, não o do resultado da função. Em um índice composto, a ordem das colunas importa: um índice sobre sobrenome e depois nome serve para uma busca só por sobrenome, mas não serve para uma busca só por nome.

O plano de execução é a ferramenta para verificar tudo isso. O comando EXPLAIN mostra como o banco pretende resolver a consulta, e nele que se vê a diferença entre uma varredura completa da tabela e um acesso por índice.